

Q1 — MCP tool call vs regular API call

A regular API call is something I write explicitly in my code — I pick the endpoint, I build the request, I call it. With an MCP tool call, I describe a set of tools to Claude and Claude decides which one to invoke and with what parameters, based on what the task actually needs. This matters because it means I don't have to anticipate every possible query path upfront — if a new question comes in, Claude can reason about which combination of tools to use rather than me having to hardcode a new flow. In practice this also means the system is more resilient to slightly different phrasings of the same question, because the decision logic lives in Claude rather than in a bunch of if-else branches I have to maintain.

Q2 — Tool call JSON structure sketch

Looking at `mcp/client.py`, when Claude Code sends a tool call to the GitHub MCP server it goes through `client.beta.messages.create` with `mcp_servers` attached and a prompt that tells Claude which tool to call. The underlying request to GitHub MCP for fetching open P0 issues would look roughly like this:

```
{
  "tool": "github_issues",
  "input": {
    "repo": "instructor/fintrack-backend-lab",
    "state": "open",
    "labels": "P0"
  }
}
```

The response comes back as a block in `response.content` with `type == "mcp_tool_result"`, and `client.py` extracts and JSON-parses it. The key thing is that MCP wraps the tool call in a structured envelope — it's not a raw HTTP call to GitHub directly, it goes through the MCP server which handles auth and translates to the GitHub REST API.

Q3 — Why read-only PAT?

The principle of least privilege — this system only needs to read data, so we should only grant it read access. If we gave Claude's MCP session a write-enabled token and that

token was compromised (leaked in logs, exposed in an env dump, intercepted), an attacker could do significant damage: they could push malicious code to the repo, close or modify existing issues, delete branches, or merge pull requests. Given that this is connected to a codebase for a financial SaaS company serving 800 enterprise clients, that blast radius could include pushing a backdoor into production code or covering up an audit trail. Read-only limits the worst case to information leakage rather than code tampering.

Q4 — BaseWorkflow.run() pattern

run() wraps execute() with three things: a console log message saying the workflow has started, a perf_counter timer measuring wall-clock time, and a try/except that catches any exception from execute(), logs it with the elapsed time, and re-raises. This pattern is useful for governance because it means every workflow automatically gets consistent timing and error logging without each subclass having to implement it themselves. It also creates a clear separation — execute() is pure business logic, and run() is the operational wrapper — so if we ever want to add audit hooks, retries, or alerts at the workflow level, we only touch base.py.

Q5 — AuditLogger fields and input_hash

Every audit entry needs: timestamp (when the call happened), workflow (which workflow triggered it), tool (which MCP tool was called), input_hash (a fingerprint of the input), status (success or error), and duration_ms (how long it took). The reason we store input_hash instead of the raw input is that tool inputs can contain sensitive data — a query might reference account IDs, internal service names, or database credentials passed as parameters. We still want to be able to tell if the same input was passed multiple times (the hash is deterministic), but we don't want the audit log itself to become a data leak. SHA-256 is one-way, so it proves the call happened without exposing what was in it.

Q6 — FINTRACK_PG_READ_URL naming

Naming it PG_READ_URL is an intentional reminder that this database connection string should only ever be used for read queries — it's a read-only PostgreSQL user. If the codebase accidentally tried to do a write through this connection, it would fail at the database level because the user has no write permissions. If a future query genuinely required write access, I'd need to add a separate FINTRACK_PG_WRITE_URL environment variable, create a dedicated write-permission database user for it, and make that very deliberate — not just reuse the existing connection. The naming convention enforces the principle in the config layer before it even reaches the database.

Q7 — Incident triage — why two DB queries?

The first DB query (window_minutes=30) establishes a baseline — what has the error rate looked like over the past half hour on average? The second query (window_minutes=5) captures the current state — what is the error rate right now? By the time Claude makes the second call, it has already seen the recent commits and open bugs, so it has context to interpret the numbers differently. For example, if the first call shows a steady 0.2% rate and the second shows 1.8%, Claude can correlate that spike with a specific deploy from the commit data. If we called both DB queries first, Claude would have the two numbers but not the commits; it couldn't make the causal link. The sequence matters.

Q8 — Graceful degradation pseudocode

```
function execute_with_degradation(service_name):
```

```
    data = {}
```

```
    try:
```

```
        data["commits"] = github_tools.search_recent_commits(service_name)
```

```
    except GitHubUnavailable:
```

```
        log_warning("GitHub MCP unreachable — skipping commits")
```

```
        data["commits"] = []
```

```
    try:
```

```
        data["issues"] = github_tools.get_priority_issues(service_name)
```

```
    except GitHubUnavailable:
```

```
        log_warning("GitHub MCP unreachable — skipping issues")
```

```
        data["issues"] = []
```

```
    # DB is considered more critical — if both DB calls fail, escalate immediately
```

```
    try:
```

```
        data["rate_30min"] = db_tools.get_error_rate(service_name, 30)
```

```
        data["rate_now"] = db_tools.get_error_rate(service_name, 5)
```

```
    except DBUnavailable:
```

```
log_error("DB MCP unreachable — cannot determine error rate")

return build_fallback(service_name, reason="DB unavailable")

# Build prompt with whatever data we have — note missing sources in the prompt
prompt = build_prompt(data, missing_sources=["github"] if not data["commits"] else
[])

response = claude.ask(prompt)

result = parse_json(response)

if data["commits"] == [] and data["issues"] == []:
    result["likely_cause"] += " (Note: GitHub data unavailable — diagnosis may be
incomplete)"

    result["escalate"] = True # Escalate when we can't fully diagnose

return result
```